

Roberto Mijat,
Software Engineer

Roberto Mijat,
Software Engineer



- Introduction to the ARM11 MPCore processor
- Introduction to MPCore processor programming
- Case study
 - MPCore processor key aspects
 - How to Initialize the ARM11 MPCore processor
 - Basic synchronization principles
 - Library API
 - Implementation recommendations
 - IPI
 - Threading via a library
 - Development environment
 - Conclusions



- Up to 4 ARMv6K processors
- Snoop control unit
- Distributed interrupt controller
- AMBA® 3 AXI™ high-speed level-two interfaces
- Each CPU features:
 - Private timer and watchdog
 - L1 caches
 - D-cache WB/WA
 - VFP support
 - JTAG based debug
 - 8 stage pipeline
 - Coprocessors 14 and 15



Multi Processing (MP)

"Execution of multiple concurrent software processes in a system"

- Key aspects:
 - Processor symmetry (SMP vs. AMP)
 - Thread
 - Sub-program with its own hardware state (and stack)
 - Synchronization
 - Establish order of execution (enforce sequentiality)
 - Protect critical sections (mutual exclusion)
 - Atomicity
 - Uninterrupted memory operation
 - Requires hardware support
 - Coherence
 - Maintain memory consistency between all CPUs



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 4



The Lightweight SMP Kernel

- Initialization (boot) code
 - Brings the MPCore processor (all CPUs) to a known/consistent state
- Distributed interrupt controller
 - Initialize interrupt distributor and CPU interfaces
 - Initialize local timers
- Scheduler
 - Round robin
 - ISR which perform a context switch between current and waiting task
 - Queues for waiting/running/terminated threads



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 5



The Lightweight SMP Kernel

- Synchronization and thread management APIs
 - Implements subset of POSIX thread API
 - Provides synchronization primitives
 - Mutexes
 - Semaphores
 - Condition variables
 - Thread creation and destruction



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 6



Initializing the MPCore Processor

- Disable Memory System
 - Disable MMU
 - D-cache must be disabled
 - Invalidate D & I caches
 - Invalidate TLBs
- Set up Auxiliary Control Register
 - CPU partaking coherency? (bit 5)
 - Configure program flow
 - Instruction Folding (bit 3)
 - Static Branch Prediction (bit 2)
 - Dynamic Branch Prediction (bit 1)
 - Return Stack (bit 0)

```

MRC p15, 0, r0, c1, c0, 0
BIC r0, r0, #0x1
BIC r0, r0, #0x4
MCR p15, 0, r0, c1, c0, 0

MOV r0, #0
MCR p15, 0, r0, c7, c7, 0
MCR p15, 0, r0, c8, c7, 0

MRC p15, 0, r0, c1, c0, 1
ORR r0, r0, #1<<5
ORR r0, r0, #1<<3
ORR r0, r0, #1<<2
ORR r0, r0, #1<<1
ORR r0, r0, #1<<0
MCR p15, 0, r0, c1, c0, 1
    
```



Auxiliary Control Register format

Initializing the MPCore Processor

- Install MMU page table
 - Register contains the physical address of first entry
- Set domain access
 - Choice between:
 - Manager
 - Reserved
 - Client
 - No access

```

; Install page table
LDR R0,=MMU_PageTable
MCR p15, 0, r0, c2, c0, 0

; Set Manager access for all domains
LDR r0,=0xFFFFFFFF
MCR p15, 0, r0, c3, c0, 0
    
```



Domain Access Control Register format

Initializing the MPCore Processor

- Configure control register
 - Enable:
 - MMU (bit 0)
 - D-cache (bit 2)
 - Write buffer (bit 3)
 - Branch prediction (bit 11)
 - I-cache (bit 12)
 - Extended page tables (bit 23)

```

MRC p15, 0, r1, c1, c0, 0
ORR r1, r1, #1
ORR r1, r1, #(1 << 2)
ORR r1, r1, #(1 << 3)
ORR r1, r1, #(1 << 11)
ORR r1, r1, #(1 << 12)
ORR r1, r1, #(1 << 23)
MCR p15, 0, r1, c1, c0, 0
    
```



Control Register format

Initializing the MPCore Processor

Configure snoop control unit

Done by one CPU only:

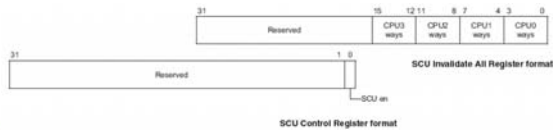
- Invalidate SCU
- Enable SCU

```

MRC    p15, 0, r1, c0, c0, 5
AND    r1, r1, #0xf
CMP    r1, #0
BNE    go_smp

MVN    r0, #0
LDR    r1, =SCU_CONTROL_REG
STR    r0, [r1, #0xc]

MOV    r0, #1
STR    r0, [r1, #0x0]
go_smp
    
```

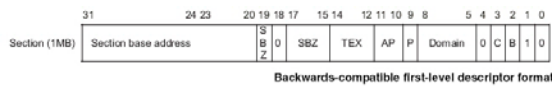


THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 10



Initializing the MPCore Processor



Backwards-compatible first-level descriptor format

```

; Definitions for section descriptors
GLOBAL EQU (1<<17)
SHARED EQU (1<<16)
APX EQU (1<<15)
TEX1 EQU (1<<12)
AP0 EQU (1<<10)
AP1 EQU (1<<11)
PARITY EQU (1<<9)
XN EQU (1<<4)
CACHE EQU (1<<3)
BUFFER EQU (1<<2)
SECTION EQU 2
MEMORY EQU (TEX1 | CACHE | BUFFER | SECTION | AP0 | AP1)
SHARED_MEMORY EQU (MEMORY | SHARED)
SHARED_SO_MEMORY EQU (SHARED | SECTION | AP0 | AP1)
SHARED_DEVICE EQU (BUFFER | SHARED | SECTION | AP0 | AP1)
    
```



THE ARCHITECTURE FOR THE DIGITAL WORLD®

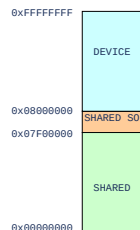
Confidential 11



Initializing the MPCore Processor

Page tables initialization

- 1Mb sections
- RAM is shared weakly ordered
- 1Mb left for strongly ordered



```

AREA PageTable, DATA, READONLY,
ALIGN=14
MMU_PageTable
GBLA count16

; first-level descriptors
; all of them are 1MB sections

count16 SETA 0
WHILE count16 < 0x1
; 0-1MB Shared Memory
DCD (count16<<20) | SHARED_MEMORY
count16 SETA count16 + 1
WEND

WHILE count16 < 0x2
; 1-2MB Strongly Ordered
DCD (count16<<20) |
SHARED_SO_MEMORY
count16 SETA count16 + 1
WEND

WHILE count16 < 0x1000
; rest of memory is Device
DCD (count16<<20) | SHARED_DEVICE
count16 SETA count16 + 1
WEND
    
```



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 12



Initializing the MPCore Processor

- Setting up the stacks
 - Per CPU
 - 4 CPUs
 - Example:
 - Per mode
 - SVC and IRQ only
- If using the ARM C Library
 - Override __user_initial_stackheap()

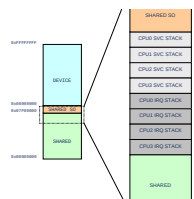


THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 13



Initializing the MPCore Processor



```

IRQ_MODE      EQU 18
SVC_MODE      EQU 19

STACK_BITS_PER_CPU EQU 14
SCV_STACK_START EQU (0x07F00000)
IRQ_STACK_START EQU (0x07F00000 -
    (0x01 << STACK_BITS_PER_CPU) * NUM_CPUS)

[...]

; which CPU are we?
MRC    p15, 0, r0, c0, 5
AND    r0, r0, #0xf

; Set up SVC stack pointer for each CPU
LDR    sp, = SCV_STACK_START
SUB    sp, sp, r0, lsl #STACK_BITS_PER_CPU

; Set up IRQ stack pointer for each CPU
CPSID  aif, #IRQ_MODE
LDR    sp, =(top_of_stacks - 0x00100000)
SUB    sp, sp, r0, lsl #STACK_BITS_PER_CPU
CPS    #SVC_MODE

[...]
    
```

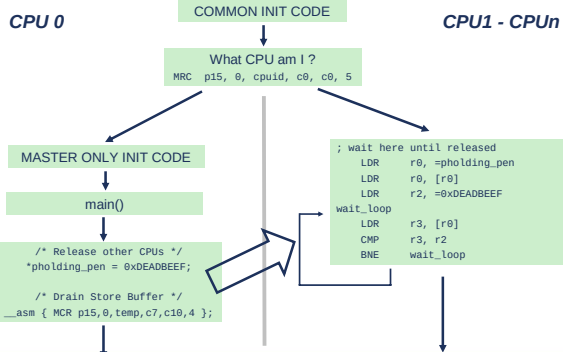


THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 14



Booting Multiple Processors



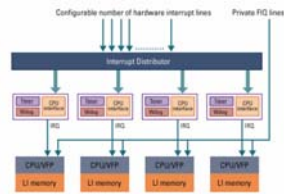
THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 15



Distributed Interrupt Controller

- Centralized independent functional block
- Interrupt Distributor
 - Role:
 - Detection of interrupts
 - Prioritization of interrupts
- CPU interfaces
 - One per CPU
 - Role:
 - Acknowledgement of interrupts,
 - masking of interrupts,
 - completion acknowledgement of interrupts



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 16

ARM

Distributed Interrupt Controller

- Software models:
 - 1-N:** interrupt taken by a CPU clears the pending status on all CPUs
 - N-N:** all CPUs receive the interrupt independently
 - deprecated
- Interrupt sources:
 - H/W interrupt
 - Private timer or watchdog interrupt
 - Legacy nIRQ pin interrupt
 - Interprocessor interrupts (IPIs) – very important for SMP OSs



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 17

ARM

Distributed Interrupt Controller

- Initializes distributor
 - Master CPU only
 - Before releasing slave CPUs
 - Interrupts disabled
 - Set targets
 - Set priorities
 - Configure (1-N model)
 - Enable
- Initialize CPU interfaces
 - All CPUs do this locally
- Initialize timer
- Install timer
- Enable interrupts

```
int main(void)
{
    int cpuid, i;

    READ_CPU_NUM(cpuid);

    if (cpuid == 0)
    {
        interrupt_distributor_init();
    }

    interrupt_interface_init();
    timer_init();

    timer_request(1, scheduler_handler, 0, 0);
    interrupts_enable();

    /* CPU0 can now release the other CPUs */
    if (cpuid == 0)
    {
        *pholding_pen = 0xdeadbeef;
        data_write_barrier();
    }

    [...]
}
```



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 18

ARM

Consistency Models

- Consistency models for memory regions:
 - Strongly ordered
 - Memory modification by a CPU immediately visible to other CPUs
 - Weakly ordered
 - Stores by one CPU can appear out of order by other CPUs
- To control **weakly ordered** model's side effects:
 - Data synchronization barrier
 - Data memory barrier
 - Flush prefetch buffer



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 19



Consistency Models

- To control **weakly ordered** model's side effects:
 - Data synchronization barrier
 - Waits for all pending explicit R/W to complete
 - All cache, branch prediction and TLB maintenance completion
 - MCR p15, 0, r0, c7, c10, 4
 - Data memory barrier
 - Ensures the apparent order of the explicit memory operations before and after it, without ensuring their completion
 - MCR p15, 0, r0, c7, c10, 5
 - Flush prefetch buffer
 - Required for cached memory region used for DMA
 - High(est) level of memory coherency
 - MCR p15, 0, r0, c7, c5, 4



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 20



Supporting Threads

- Key points:
 - Critical sections
 - Piece of code that can only be executed by one thread at a time
 - Protected using mutexes
 - Serialization
 - Synchronize order of execution
 - Semaphores (counting mutexes) and condition variables
 - Timeslicing
 - Scheduler automatically reschedules – round robin
 - Timer ticks on each CPU – determines time-slice
 - Auto-reload timer which triggers IRQ – ISR handles rescheduling



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 21



Supporting Threads

- Key points:
 - Context switching
 - PCBs management
 - Working registers (R0 to R12)
 - PC, SP, LR, CPSR
 - VFP registers (if VFP enabled)
 - SRS/RFE
 - CPSID/CPSIE



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 22

ARM

Supporting Threads

- Basic threading API based on POSIX Threads
 - Selected subset supporting:
 - Exclusive access
 - `pthread_mutex_init()`
 - `pthread_mutex_lock()`
 - `pthread_mutex_unlock()`
 - `pthread_mutex_destroy()`
 - Synchronization
 - `pthread_cond_init()`
 - `pthread_cond_signal()`
 - `pthread_cond_broadcast()`
 - `pthread_cond_wait()`
 - `pthread_cond_destroy()`
 - `sem_init()`, `sem_post()`, `sem_wait()`, `sem_destroy()`
 - Thread management
 - `pthread_create()`
 - `pthread_join()`



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 23

ARM

Architectural Support for Primitives

- To support extensions to the ARMv6 architecture, the MPCore processor includes additional instructions:
 - Wait for Event (WFE) / Send Event (SEV)
 - Load Exclusive (LDREX) / Store Exclusive (STREX)
- Wait For Event (WFE)
 - If the event register is currently set
 - Clears it and returns immediately
 - If the event register is not set
 - Suspend execution (Clock is stopped) until:
 - IRQ/FIQ (unless I/F masked in the CPSR)
 - A debug entry request made (and debug is enabled)
 - An event is signalled by another processor using Send Event.
 - An MPCore CPU return from exception
- Send Event (SEV)
 - Signal event to all CPUs



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 24

ARM

Locks Implementation

- For ARMv5 architecture and below:
 - SWP instruction
 - Atomically swap a word in memory with a register
 - Hold external bus until completion (unless coherent region)
 - Deprecated

```

LOCK      EQU      1
UNLOCK    EQU      0

lock
    MOV    r0, #LOCK    ; value to claim the
loop
    SWP    r0, r0, [r1]  ; try and claim the
lock
    CMP    r0, #LOCK    ; check if it was
already claimed
    BEQ    loop          ; if so wait for it
to become free
    
```



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 25



Locks Implementation

- For ARMv6 architecture

```

LOCK      EQU      1
UNLOCK    EQU      0

spin_lock
    MOV    r2, #LOCK
spin
    LDREX  r1, [r0]      ; see what state the lock is in
    TEQ    r1, #UNLOCK   ; locked already?
    WFE    ; wait if it's locked
    STREXEQ r1, r2, [r0] ; try to lock it, if it's unlocked
    TEQEQ  r1, #0x0      ; did we fail?
    BNE    spin          ; if any failure, loop
    MOV    pc, lr

spin_unlock
    MOV    r1, #UNLOCK
    STR    r1, [r0]      ; release spin lock
    MCR    p15, 0, r1, c7, c10, 4 ; Drain Store Buffer (DSB)
    SEV    ; unlock any spinlock
    MOV    pc, lr
    
```



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 26



Interprocessor Interrupts (IPIs)

- SMP OSs require communication between CPUs
 - Best without accessing memory
 - Cause activity on a remote processor
 - Need to synchronize asynchronously
- Each CPU has private interrupts (ID0-ID15)
 - Software triggered only
 - Aliased for each CPU
- Broadcast modes
 - All
 - All-but-self
 - Self
 - A specific other CPU
- Example:
 - An application allocates some virtual memory
 - OS needs to apply these translations to other processors
 - Cause IRQ on other CPUs which in turn will update their tables



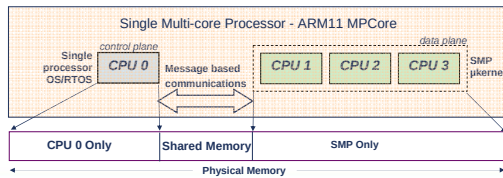
THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 27



Threading via a Library

- Design principle:
 - Master CPU
 - Single processor OS + application + driver for off-loaded API
 - Slave CPUs
 - Wait and/or process background worktasks
 - Master CPU driver abstracts shared RAM and provides message based API to data plane processor farm



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 28



Threading via a Library: Linux Example

- Reserve **protected** area in memory
 - load SMP kernel in it
- Normal SMP Linux boot:
 - CPU0 start booting
 - CPU1-n are in WaitForInterrupt state
 - Point secondary boot address to start of kernel
 - Send IPI to CPU1-n (which break WFI – IRQs are disabled)
 - kernel boots up and is now ready (CPU1-n)
- Device implemented for communication between Linux and kernel
 - /dev/ourOS/in_and_out



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 29



Development Environment

- MPCore processor testchip on development board
- RealView ICE JTAG run control hardware unit
 - Debug agent
- RealView Development Suite
- RealView debugger
 - Multi-core debug



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 30



Conclusions

- We have seen
 - How to boot the MPCore processor
 - MMU, SCU, caches, stacks
 - How to implement MPCore primitives (low level)
 - Mutexes, semaphores etc.
 - Architecture support (WFE/SEV, STREX/LDREX etc.)
 - Coherence (barriers etc.)
 - Support for threads
 - Scheduler principles
 - Time-slicing
 - Context switching



THE ARCHITECTURE FOR THE DIGITAL WORLD®

Confidential 31